Panduan Penyiapan Server

Diterjemahkan dari `docs/final/server-setup.md` pada 2 November 2026. Perintah, berkas konfigurasi, dan komentar di dalam blok kode dibiarkan apa adanya.

- **Status:** final (disetujui 2026-10-31, D-123)
- **Tugas:** P2-05
- **Terakhir diperbarui:** 2026-10-31 (jawaban pemilik: domain, VPS, R2, staging, Ubuntu 24.04, 8 vCPU / 16 GB, Cloudflare Tunnel, disk 320 GB, staging di VPS yang sama)
- **Versi dicek daring pada 2026-10-30** (sumber di bagian Versi).

Tujuan

Panduan langkah demi langkah untuk menyiapkan VPS SIGRAK (IDCloudHost Cloud VPS, Jakarta), yang menjalankan produksi dan, sebagai situs kedua, staging: aplikasi utama, back office Filament, situs ujian TKA, worker antrean, dan layanan PDF Gotenberg. VPS ini **tidak punya IP publik**: semua lalu lintas web dan SSH masuk lewat Cloudflare Tunnel (`cloudflared`), yang hanya membuat koneksi keluar. Panduan ini untuk siapa pun yang menyiapkan atau membangun ulang server. Jalankan bagian-bagiannya berurutan. Setiap perintah dimaksudkan untuk disalin apa adanya. Ganti hanya placeholder di bawah; jangan pernah meng-commit nilai sungguhan.

Placeholder	Arti
`app.sigrak.com`	Aplikasi sekolah
`ujian.sigrak.com`	Situs ujian TKA (D-027)
`admin.sigrak.com`	Konsol staf (Filament) + Horizon
`ssh.sigrak.com`	SSH lewat tunnel, di balik Cloudflare Access
`<TUNNEL_ID>`	UUID yang dicetak oleh `cloudflared tunnel create`
`<TEAM_EMAILS>`	Email anggota tim yang diizinkan oleh Cloudflare Access
`<STAGING_DB_PASSWORD>`	Kata sandi MySQL untuk pengguna database staging
`<DB_PASSWORD>`, `<DB_BACKUP_PASSWORD>`, `<MYSQL_ROOT_PASSWORD>`	Kata sandi MySQL
`<REDIS_PASSWORD>`	Kata sandi Redis
`<GOTENBERG_USER>`, `<GOTENBERG_PASSWORD>`	Basic auth Gotenberg
`<R2_ACCOUNT_ID>`, `<R2_ACCESS_KEY_ID>`, `<R2_SECRET_ACCESS_KEY>`	Account ID Cloudflare R2 dan token API yang dibatasi ke bucket
`<R2_BUCKET>`	Bucket R2 untuk cadangan (nama yang disarankan: `sigrak-backups`)
`<BACKUP_CRYPT_PASSWORD>`, `<BACKUP_CRYPT_SALT>`	Kata sandi crypt rclone (mengenkripsi cadangan sebelum diunggah)

Buat setiap rahasia dengan `openssl rand -base64 32 | tr -d '/+=' | cut -c1-32` dan simpan di pengelola kata sandi tim.

Hostname, guard, dan penanganan waktu mengikuti `architecture.md` (P2-04): satu codebase Laravel, tiga kelompok yang dirutekan per domain, cookie sesi host-only, server dalam UTC, sedangkan aplikasi dan sesi MySQL-nya berjalan dalam WIB (`Asia/Jakarta`, `+07:00`, D-165).

Versi

Dicek pada 2026-10-30. "Kunci" berarti versinya ditetapkan di konfigurasi dan hanya diubah dengan sengaja.

Komponen	Pilihan	Alasan	Dukungan (saat dicek)	Sumber
OS	**Ubuntu Server 24.04 LTS** (pemilik, 2026-10-31)	Dukungan standar 22.04 berakhir pertengahan 2027, tepat setelah musim TKA pertama	Dukungan standar sampai pertengahan 2029	Daftar rilis di ubuntu.com
Laravel	13.x	Versi utama terkini (13.0 pada 2026-03-17)	Perbaikan bug sampai Q3 2027, keamanan sampai 2028-03-17	laravel.com/docs/13.x/releases
PHP	**8.5** (≥ 8.5.12) dari `ppa:ondrej/php`	Laravel 13 mendukung 8.3–8.5; 8.5 punya dukungan aktif sampai 2027-12-31, keamanan sampai 2029-12-31. 8.5.12 memperbaiki CVE Okt 2026	php.net/supported-versions	php.net, launchpad.net/~ondrej
Nginx	Paket Ubuntu 24.04 (1.24 + patch keamanan Ubuntu), hanya mendengar di localhost	Lebih sedikit komponen; ditambal Canonical	Mengikuti OS	Arsip Ubuntu
MySQL	**8.4 LTS** (8.4.13 pada 2026-10-20) dari repo.mysql.com, komponen `mysql-8.4-lts`	MySQL 8 sesuai D-007; 8.4 adalah jalur LTS 8.x	Premier support sampai Apr 2029	Catatan rilis dev.mysql.com, endoflife.date/mysql
Redis	**8.2.x** dari packages.redis.io, dikunci `6:8.2.*` (8.2.11 untuk noble)	8.2 adalah rilis extended (perbaikan 5 tahun); 8.10 paling baru tetapi berumur pendek	Sampai 2030-09-01	redis.io, packages.redis.io
Horizon	Versi utama terbaru lewat Composer	Dasbor antrean + supervisor di Redis	—	laravel.com/docs/13.x/horizon
Filament	5.x	PHP 8.2+, Laravel 11.28+	—	filamentphp.com/docs/5.x
Laravel Excel	`maatwebsite/excel` 4.x	4.x mendukung Laravel 12/13 dan PHP 8.5 (3.1 menghalangi PHP 8.5 lewat PhpSpreadsheet 1.x)	—	packagist.org/packages/maatwebsite/excel
Docker Engine	docker-ce dari download.docker.com	Hanya untuk Gotenberg (D-028)	—	docs.docker.com

Komponen	Pilihan	Alasan	Dukungan (saat dicek)	Sumber
Gotenberg	**`gotenberg/gotenberg:8.39.0-chromium`** (rilis 2026-10-27)	Varian khusus Chromium: tanpa LibreOffice (tidak ada konversi Word ke PDF, D-031), ~30% lebih kecil	—	github.com/gotenberg/gotenberg/releases, Docker Hub
Node.js	**24 LTS** (24.22.2), hanya untuk build	LTS pemeliharaan sampai 2028-04-30, matang dengan Vite/Tailwind. Node 26 menjadi LTS pada 2026-10-28; ditinjau lagi setelah stabil	—	Rilis nodejs.org, jadwal nodejs/Release
Cloudflare Tunnel	`cloudflared` **2026.10.7** dari pkg.cloudflare.com (tunnel yang dikelola lokal)	VPS tanpa IP publik; hanya koneksi keluar di port 7844	—	developers.cloudflare.com/tunnel, pkg.cloudflare.com
Cloudflare	Paket gratis: DNS, proxy, WAF, Zero Trust Access (gratis sampai 50 pengguna)	CONTEXT.md	—	developers.cloudflare.com

Isi

1. Sistem dasar

Buat VM di konsol IDCloudHost: **Cloud VPS, lokasi Jakarta**, image **Ubuntu Server 24.04 LTS**, paket kustom **8 vCPU / 16 GB RAM / disk NVMe 320 GB**. **Jangan** pasang IP publik (floating). VM tetap butuh internet keluar (NAT) untuk apt, GitHub, Gemini, R2, dan tunnel itu sendiri (TCP dan UDP port 7844 ke Cloudflare); sudah dipastikan oleh IDCloudHost (2026-10-31).

Tanpa IP publik, langkah-langkah pertama dijalankan di **konsol web IDCloudHost** (layar VM di browser). Masuk dengan pengguna dan kata sandi yang diatur saat membuat VM, lalu pindah ke root (`sudo -i`):

```
apt update && apt -y full-upgrade
apt -y install curl gnupg ca-certificates lsb-release software-properties-common \
  unzip git acl ufw unattended-upgrades supervisor logrotate
timedatectl set-timezone Etc/UTC   # the app runs in WIB (Asia/Jakarta); cron times below are UTC
hostnamectl set-hostname sigrak-prod
dpkg-reconfigure -plow unattended-upgrades   # answer "Yes"
curl -sS -m 10 -o /dev/null -w '%{http_code}\n' https://api.cloudflare.com/client/v4/   # any HTTP
code = outbound works
```

Swap (4 GB), agar lonjakan job PDF tidak memicu OOM killer:

```
fallocate -l 4G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
echo 'vm.swappiness=10' > /etc/sysctl.d/99-swappiness.conf && sysctl --system
```

Pengguna: satu pengguna admin dengan sudo untuk manusia, dan pengguna `sigrak` yang memiliki kode serta menjalankan PHP-FPM, Horizon, dan scheduler.

```
adduser --gecos "" admin && usermod -aG sudo admin
mkdir -p /home/admin/.ssh
echo 'ssh-ed25519 AAAA... you@laptop' >> /home/admin/.ssh/authorized_keys   # paste each admin's
public key
chown -R admin:admin /home/admin/.ssh && chmod 700 /home/admin/.ssh && chmod 600
/home/admin/.ssh/authorized_keys

adduser --system --group --home /var/www/sigrak --shell /bin/bash sigrak
usermod -aG sigrak www-data      # Nginx reads public/ through the group
```

Pengamanan SSH. SSH hanya bisa dijangkau lewat tunnel (bagian 9), dan konsol web IDCloudHost bukan SSH, jadi pengaturan ini tidak bisa mengunci Anda di luar.

```
cat > /etc/ssh/sshd_config.d/99-sigrak.conf <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
sshd -t && systemctl reload ssh
```

fail2ban tidak dipakai: setiap koneksi SSH datang dari `cloudflared` di `127.0.0.1`, yang tidak pernah diblokir fail2ban, dan Cloudflare Access sudah memeriksa siapa yang boleh tersambung.

Mulai dari sini, bekerja sebagai `admin` dengan `sudo` (di konsol sampai bagian 9, lalu lewat SSH melalui tunnel).

2. Firewall (UFW)

Tidak ada yang diterima dari luar: tidak ada IP publik, dan `cloudflared` menjangkau Nginx serta SSH lewat `127.0.0.1`, yang selalu diizinkan UFW. UFW tetap menjaga jaringan privat penyedia.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose     # Status: active, no ALLOW rules
```

Docker menulis aturan iptables sendiri dan melewati UFW untuk port yang dipublikasikan. Karena itu Gotenberg hanya dipublikasikan di `127.0.0.1` (bagian 7); jangan pernah mempublikasikannya sebagai `3000:3000`.

3. PHP 8.5 + PHP-FPM

```
sudo LC_ALL=C.UTF-8 add-apt-repository -y ppa:ondrej/php
sudo apt update
sudo apt -y install php8.5-fpm php8.5-cli php8.5-common php8.5-mysql php8.5-redis \
  php8.5-mbstring php8.5-xml php8.5-curl php8.5-zip php8.5-gd php8.5-intl php8.5-bcmath
php -v    # must show 8.5.12 or later
```

PHP 8.5 selalu menyertakan OPcache; tidak ada paket `php8.5-opcache` terpisah. Ekstensi ini mencakup Laravel, Laravel Excel (zip, xml, gd, iconv), PhpWord (zip, xml), dan Horizon (redis/pcntl, pcntl sudah ada di CLI).

Pengaturan PHP untuk FPM dan CLI:

```
sudo tee /etc/php/8.5/fpm/conf.d/99-sigrak.ini > /dev/null <<'EOF'
expose_php = Off
```

```
memory_limit = 256M
upload_max_filesize = 20M
post_max_size = 25M
max_execution_time = 60
date.timezone = UTC
opcache.enable = 1
opcache.memory_consumption = 256
opcache.interned_strings_buffer = 16
opcache.max_accelerated_files = 20000
opcache.validate_timestamps = 0
EOF
sudo tee /etc/php/8.5/cli/conf.d/99-sigrak.ini > /dev/null <<'EOF'
date.timezone = UTC
EOF
```

`opcache.validate_timestamps = 0` berarti PHP-FPM harus di-reload pada setiap deploy (bagian 11 melakukannya).

Pool FPM yang berjalan sebagai `sigrak`:

```
sudo tee /etc/php/8.5/fpm/pool.d/sigrak.conf > /dev/null <<'EOF'
[sigrak]
user = sigrak
group = sigrak
listen = /run/php/php8.5-fpm-sigrak.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660
pm = dynamic
pm.max_children = 60
pm.start_servers = 8
pm.min_spare_servers = 4
pm.max_spare_servers = 12
pm.max_requests = 500
request_terminate_timeout = 65s
EOF
sudo mv /etc/php/8.5/fpm/pool.d/www.conf /etc/php/8.5/fpm/pool.d/www.conf.disabled
sudo systemctl restart php8.5-fpm && sudo systemctl enable php8.5-fpm
```

`pm.max_children` ≈ (RAM yang tersisa untuk PHP) ÷ (rata-rata ukuran worker, sekitar 60–80 MB). 60 (sekitar 4–5 GB) muat di VPS 16 GB berdampingan dengan MySQL, Redis, Horizon, dan Gotenberg. Periksa dengan `ps -o rss= -C php-fpm8.5 | sort -n | tail -3`.

Composer (installer dicocokkan dengan tanda tangan yang dipublikasikan):

```
cd /tmp
EXPECTED="$(curl -fsSL https://composer.github.io/installer.sig)"
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
ACTUAL="$(php -r "echo hash_file('sha384', 'composer-setup.php');")"
if [ "$EXPECTED" = "$ACTUAL" ]; then
  sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer
else
  echo "Composer installer checksum mismatch, not installing"
fi
rm -f composer-setup.php
composer --version
```

4. MySQL 8.4 LTS

Ubuntu 24.04 membawa MySQL 8.0 (sudah habis masa dukungannya sejak April 2026), jadi pakai repositori APT Oracle dengan komponen `mysql-8.4-lts`, yang tidak pernah melompat sendiri ke 9.x.

```
curl -fsSL https://repo.mysql.com/RPM-GPG-KEY-mysql-2025 | sudo gpg --dearmor -o
/usr/share/keyrings/mysql.gpg
echo "deb [signed-by=/usr/share/keyrings/mysql.gpg] http://repo.mysql.com/apt/ubuntu $(lsb_release -
cs) mysql-8.4-lts" \
  | sudo tee /etc/apt/sources.list.d/mysql.list
sudo apt update
apt-cache policy mysql-server     # candidate must be 8.4.x from repo.mysql.com
sudo apt -y install mysql-server   # set <MYSQL_ROOT_PASSWORD> when asked; choose strong password
encryption
```

Pengaturan server:

```
sudo tee /etc/mysql/mysql.conf.d/zz-sigrak.cnf > /dev/null <<'EOF'
[mysqld]
bind-address = 127.0.0.1
mysqlx = OFF
character-set-server = utf8mb4
collation-server = utf8mb4_0900_ai_ci
default-time-zone = '+00:00'
innodb_buffer_pool_size = 5G
max_connections = 200
slow_query_log = ON
slow_query_log_file = /var/log/mysql/slow.log
long_query_time = 1
EOF
sudo systemctl restart mysql && sudo systemctl enable mysql
```

`innodb_buffer_pool_size`: sekitar 30% RAM di server bersama ini (5G dari 16 GB).

Database, pengguna aplikasi, dan pengguna cadangan yang hanya-baca:

```
sudo mysql -uroot -p <<'EOF'
CREATE DATABASE sigrak CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE USER 'sigrak'@'localhost' IDENTIFIED BY '<DB_PASSWORD>';
GRANT ALL PRIVILEGES ON sigrak.* TO 'sigrak'@'localhost';
CREATE USER 'backup'@'localhost' IDENTIFIED BY '<DB_BACKUP_PASSWORD>';
GRANT SELECT, SHOW VIEW, TRIGGER, LOCK TABLES, EVENT ON sigrak.* TO 'backup'@'localhost';
FLUSH PRIVILEGES;
EOF
```

5. Redis 8.2

Ubuntu 24.04 membawa Redis 7.0, jadi pakai repositori resmi dan kunci jalur 8.2:

```
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/redis-archive-
keyring.gpg
sudo chmod 644 /usr/share/keyrings/redis-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] https://packages.redis.io/deb
$(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/redis.list
sudo tee /etc/apt/preferences.d/redis > /dev/null <<'EOF'
Package: redis redis-server redis-sentinel redis-tools
Pin: version 6:8.2.*
Pin-Priority: 1001
```

```
EOF
sudo apt update
sudo apt -y install redis redis-server redis-tools
redis-server --version    # must show 8.2.x
```

Pengaturan. `noeviction` wajib: data antrean dan Horizon tidak boleh pernah dibuang. Karena itu entri cache selalu butuh TTL.

```
sudo tee -a /etc/redis/redis.conf > /dev/null <<'EOF'

# --- SIGRAK ---
bind 127.0.0.1 -::1
protected-mode yes
requirepass <REDIS_PASSWORD>
maxmemory 2gb
maxmemory-policy noeviction
appendonly yes
appendfsync everysec
EOF
sudo systemctl restart redis-server && sudo systemctl enable redis-server
redis-cli -a '<REDIS_PASSWORD>' --no-auth-warning ping    # PONG
```

Redis 8 berlisensi AGPLv3/RSALv2/SSPLv1. Kita menjalankannya tanpa diubah sebagai layanan internal, dan tidak ada lisensi itu yang membatasinya.

6. Nginx

Nginx hanya mendengar di `127.0.0.1:8080`, dengan HTTP biasa. TLS berakhir di edge Cloudflare, dan lompatan dari Cloudflare ke server berjalan di dalam tunnel yang terenkripsi. Pengaturan **Always Use HTTPS** di Cloudflare (bagian 9) yang mengalihkan HTTP → HTTPS.

```
sudo apt -y install nginx
sudo rm -f /etc/nginx/sites-enabled/default
sudo sed -i 's/# server_tokens off;/server_tokens off;/' /etc/nginx/nginx.conf
```

IP klien sebenarnya: setiap request datang dari `cloudflared` di localhost, membawa alamat pengunjung di `CF-Connecting-IP`. Nginx memasang alamat itu, sehingga log Laravel, pembatasan laju, dan jejak audit ujian melihat IP sebenarnya.

```
sudo tee /etc/nginx/conf.d/realip.conf > /dev/null <<'EOF'
set_real_ip_from 127.0.0.1;
set_real_ip_from ::1;
real_ip_header CF-Connecting-IP;
EOF
```

Snippet bersama untuk ketiga vhost. `fastcgi_param HTTPS on` memberi tahu Laravel bahwa pengunjung memakai HTTPS, sehingga URL dan cookie aman sudah benar tanpa harus memercayai header proxy.

```
sudo tee /etc/nginx/snippets/sigrak-common.conf > /dev/null <<'EOF'
root /var/www/sigrak/current/public;
index index.php;
charset utf-8;

add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

```
location / {
    try_files $uri $uri/ /index.php?$query_string;
}
location /build/ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    try_files $uri =404;
}
location = /favicon.ico { access_log off; log_not_found off; }
location = /robots.txt  { access_log off; log_not_found off; }
error_page 404 /index.php;

location ~ ^/index\.php(/|$) {
    fastcgi_pass unix:/run/php/php8.5-fpm-sigrak.sock;
    fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    include fastcgi_params;
    fastcgi_param HTTPS on;
    fastcgi_hide_header X-Powered-By;
    fastcgi_read_timeout 60s;
}
location ~ /\.(?!well-known).* { deny all; }
location ~ \.php$ { return 404; }
EOF
```

Vhost aplikasi sekolah (`app.sigrak.com`):

```
sudo tee /etc/nginx/sites-available/sigrak-app.conf > /dev/null <<'EOF'
server {
    listen 127.0.0.1:8080;
    server_name app.sigrak.com;
    client_max_body_size 25m;
    access_log /var/log/nginx/sigrak-app.access.log;
    error_log  /var/log/nginx/sigrak-app.error.log;
    include snippets/sigrak-common.conf;
}
EOF
```

Vhost situs ujian (`ujian.sigrak.com`). Codebase sama, dengan log sendiri dan batas unggahan kecil (situs ujian menerima jawaban, bukan berkas). Jangan tambahkan pembatasan laju per IP di sini: satu kelas sering berbagi satu IP sekolah.

```
sudo tee /etc/nginx/sites-available/sigrak-ujian.conf > /dev/null <<'EOF'
server {
    listen 127.0.0.1:8080;
    server_name ujian.sigrak.com;
    client_max_body_size 1m;
    access_log /var/log/nginx/sigrak-ujian.access.log;
    error_log  /var/log/nginx/sigrak-ujian.error.log;
    include snippets/sigrak-common.conf;
}
EOF
```

Vhost konsol staf (`admin.sigrak.com`: Filament dan Horizon di balik guard `staff`, dan di balik Cloudflare Access, bagian 9):

```
sudo tee /etc/nginx/sites-available/sigrak-admin.conf > /dev/null <<'EOF'
server {
    listen 127.0.0.1:8080;
    server_name admin.sigrak.com;
    client_max_body_size 25m;
    access_log /var/log/nginx/sigrak-admin.access.log;
```

```
    error_log  /var/log/nginx/sigrak-admin.error.log;
    include snippets/sigrak-common.conf;
}
EOF
```

Penampung untuk nama host lain:

```
sudo tee /etc/nginx/sites-available/00-default.conf > /dev/null <<'EOF'
server {
    listen 127.0.0.1:8080 default_server;
    server_name _;
    return 444;
}
EOF
sudo ln -s /etc/nginx/sites-available/00-default.conf /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/sigrak-app.conf /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/sigrak-ujian.conf /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/sigrak-admin.conf /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
curl -s -o /dev/null -w '%{http_code}\n' -H 'Host: app.sigrak.com' http://127.0.0.1:8080/   # 404
until the first deploy, then 200 or 302
```

7. Docker + Gotenberg

Docker Engine dari repositori Docker:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt -y install docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo tee /etc/docker/daemon.json > /dev/null <<'EOF'
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
EOF
sudo systemctl restart docker && sudo systemctl enable docker
```

Gotenberg dengan image yang dikunci, terikat ke localhost, dengan basic auth, di jaringan Docker sendiri tanpa akses keluar (aplikasi mengirim setiap aset di dalam request, lihat `architecture.md`). Folder fonts berisi Plus Jakarta Sans dan font Arab (D-033) agar PDF tampil sama dengan aplikasi.

```
sudo mkdir -p /opt/gotenberg/fonts
sudo tee /opt/gotenberg/.env > /dev/null <<'EOF'
GOTENBERG_API_BASIC_AUTH_USERNAME=<GOTENBERG_USER>
GOTENBERG_API_BASIC_AUTH_PASSWORD=<GOTENBERG_PASSWORD>
EOF
sudo chmod 600 /opt/gotenberg/.env
sudo tee /opt/gotenberg/compose.yaml > /dev/null <<'EOF'
services:
  gotenberg:
    image: gotenberg/gotenberg:8.39.0-chromium
    restart: unless-stopped
    ports:
      - "127.0.0.1:3000:3000"
    env_file: .env
```

```
    command:
      - "gotenberg"
      - "--api-enable-basic-auth"
      - "--api-timeout=120s"
    volumes:
      - ./fonts:/usr/local/share/fonts/sigrak:ro
    mem_limit: 2g
    networks:
      - gotenberg

networks:
  gotenberg:
    driver: bridge
    ipam:
      config:
        - subnet: 172.31.250.0/24
EOF
# copy the font files (.ttf/.otf) into /opt/gotenberg/fonts first, then:
cd /opt/gotenberg && sudo docker compose up -d
curl -s -o /dev/null -w '%{http_code}\n' -u '<GOTENBERG_USER>:<GOTENBERG_PASSWORD>'
http://127.0.0.1:3000/health   # 200
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:3000/forms/chromium/convert/html
# 401 (auth works)
```

Blokir koneksi baru **dari** container, baik ke internet (chain `DOCKER-USER` milik Docker) maupun ke host itu sendiri (`INPUT`). Request dari Laravel ke `127.0.0.1:3000` dan balasannya tetap berjalan.

```
sudo tee /usr/local/sbin/gotenberg-egress > /dev/null <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
NET=172.31.250.0/24
for CHAIN in DOCKER-USER INPUT; do
  iptables -C "$CHAIN" -s "$NET" -m conntrack --ctstate NEW -j DROP 2>/dev/null \
    || iptables -I "$CHAIN" 1 -s "$NET" -m conntrack --ctstate NEW -j DROP
done
EOF
sudo chmod 755 /usr/local/sbin/gotenberg-egress
sudo tee /etc/systemd/system/gotenberg-egress.service > /dev/null <<'EOF'
[Unit]
Description=Block outbound traffic from the Gotenberg container
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/gotenberg-egress
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now gotenberg-egress
cd /opt/gotenberg && sudo docker compose exec gotenberg bash -c 'timeout 5 bash -c "
</dev/tcp/1.1.1.1/443" && echo OPEN || echo BLOCKED'   # BLOCKED
```

Memperbarui Gotenberg adalah perubahan yang disengaja: ubah tag-nya, `sudo docker compose pull && sudo docker compose up -d`, lalu cetak satu PDF dari setiap templat sebelum perubahan ditutup.

8. Node.js (hanya untuk build)

Disarankan: build aset front-end (`npm ci && npm run build`) di CI atau di komputer pengembang, lalu kirim `public/build/` bersama rilis. Dengan begitu server tidak butuh Node sama sekali.

Bila server harus melakukan build, pasang binary Node 24 LTS yang dikunci, dicocokkan dengan checksum resmi:

```
NODE_VERSION=v24.22.2
cd /tmp
curl -fsSLO "https://nodejs.org/dist/${NODE_VERSION}/node-${NODE_VERSION}-linux-x64.tar.xz"
curl -fsSLO "https://nodejs.org/dist/${NODE_VERSION}/SHASUMS256.txt"
grep " node-${NODE_VERSION}-linux-x64.tar.xz\$" SHASUMS256.txt | sha256sum -c -
sudo mkdir -p /opt/node
sudo tar -xJf "node-${NODE_VERSION}-linux-x64.tar.xz" -C /opt/node --strip-components=1
sudo ln -sf /opt/node/bin/node /opt/node/bin/npm /opt/node/bin/npx /usr/local/bin/
node -v
```

Python tidak dipasang untuk aplikasi. `python3` bawaan Ubuntu dibiarkan apa adanya; alat Python apa pun mendapat folder dan virtualenv sendiri bila sebuah tugas membutuhkannya (CONTEXT.md).

9. Cloudflare Tunnel dan Access

Tunnel tersambung keluar dari VPS ke Cloudflare dan membawa ketiga hostname web ditambah SSH. Tidak ada rekaman DNS yang menunjuk ke server; setiap hostname adalah CNAME yang di-proxy ke `<TUNNEL_ID>.cfargotunnel.com`.

Pasang `cloudflared` dari repositori Cloudflare (di konsol IDCloudHost):

```
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee /usr/share/keyrings/cloudflare-
main.gpg > /dev/null
echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.com/cloudflared
any main' \
  | sudo tee /etc/apt/sources.list.d/cloudflared.list
sudo apt update && sudo apt -y install cloudflared
cloudflared --version     # 2026.10.7 or later
```

Buat tunnel. `tunnel login` mencetak sebuah URL: buka di laptop, masuk ke Cloudflare, lalu pilih `sigrak.com`.

```
sudo cloudflared tunnel login                  # writes /root/.cloudflared/cert.pem
sudo cloudflared tunnel create sigrak-prod     # prints <TUNNEL_ID>, writes
/root/.cloudflared/<TUNNEL_ID>.json
sudo install -d -m 755 /etc/cloudflared
sudo install -m 600 /root/.cloudflared/<TUNNEL_ID>.json /etc/cloudflared/<TUNNEL_ID>.json
sudo tee /etc/cloudflared/config.yml > /dev/null <<'EOF'
tunnel: <TUNNEL_ID>
credentials-file: /etc/cloudflared/<TUNNEL_ID>.json
ingress:
  - hostname: app.sigrak.com
    service: http://127.0.0.1:8080
  - hostname: ujian.sigrak.com
    service: http://127.0.0.1:8080
  - hostname: admin.sigrak.com
    service: http://127.0.0.1:8080
  - hostname: ssh.sigrak.com
    service: ssh://127.0.0.1:22
  - service: http_status:404
EOF
sudo cloudflared tunnel ingress validate --config /etc/cloudflared/config.yml
for h in app ujian admin ssh; do sudo cloudflared tunnel route dns sigrak-prod "$h.sigrak.com"; done
sudo cloudflared service install               # systemd unit using /etc/cloudflared/config.yml
sudo systemctl enable --now cloudflared
sudo cloudflared tunnel info sigrak-prod       # shows active connections
```

`cert.pem` hanya dibutuhkan untuk membuat tunnel dan rute DNS, bukan untuk menjalankannya. Simpan salinannya di pengelola kata sandi dan hapus dari server: `sudo shred -u /root/.cloudflared/cert.pem`. JSON kredensial tunnel tetap di server (mode 600); perlakukan sebagai rahasia.

Bila UDP 7844 diblokir, `cloudflared` otomatis beralih ke HTTP/2 lewat TCP 7844.

Cloudflare Access (dasbor Zero Trust → Access → Applications → Add → Self-hosted):

1. `ssh.sigrak.com`: kebijakan "Allow" untuk `<TEAM_EMAILS>`. **Wajib**, agar hanya tim yang bisa menjangkau port SSH.
2. `admin.sigrak.com`: kebijakan yang sama. Disarankan: konsol staf lalu butuh login Access dan login guard `staff`.
3. Jangan pernah menaruh `app.` atau `ujian.` di balik Access: guru dan siswa masuk dengan login aplikasi sendiri.

SSH dari laptop. Pasang `cloudflared` di laptop, lalu tambahkan ke `~/.ssh/config`:

```
Host ssh.sigrak.com
  User admin
  ProxyCommand cloudflared access ssh --hostname %h
```

`ssh ssh.sigrak.com` membuka browser untuk login Access pada kali pertama, lalu tersambung dengan kunci SSH Anda. Setelah ini berhasil, kerjakan sisa penyiapan lewat SSH. Konsol web IDCloudHost tetap tersedia sebagai jalan masuk darurat bila tunnel mati.

Pengaturan zona (dasbor Cloudflare → `sigrak.com`):

- SSL/TLS → Edge Certificates: **Always Use HTTPS** aktif, **Minimum TLS Version** 1.2. Aktifkan HSTS hanya setelah seminggu tanpa masalah TLS. Mode enkripsi (Full/Strict) tidak berpengaruh pada hostname tunnel; biarkan di **Full (strict)**.
- Tidak perlu sertifikat Origin CA, Authenticated Origin Pulls, atau daftar izin IP Cloudflare: tidak ada yang bisa menjangkau server kecuali lewat tunnel.

Hal yang diubah Cloudflare untuk aplikasi:

- Request yang di-proxy habis waktu setelah 100 detik. Pembuatan konten AI dan ekspor PDF sudah berjalan sebagai job antrean; tidak boleh ada request web yang menunggunya.
- Unggahan di paket gratis dibatasi 100 MB per request (unggahan kita jauh di bawahnya).
- Biarkan mode "I'm Under Attack" dan aturan bot yang agresif **mati** untuk `ujian.sigrak.com`: satu kelas masuk pada saat yang sama dari satu IP, yang terlihat seperti serangan.
- Cloudflare tidak meng-cache HTML secara bawaan; biarkan begitu. Hanya `/build/*` yang di-cache jangka panjang.
- Bila `cloudflared` berhenti, seluruh situs offline. systemd menjalankannya ulang; perbarui setiap bulan dengan `sudo apt update && sudo apt -y install --only-upgrade cloudflared && sudo systemctl restart cloudflared`.

10. Folder aplikasi, environment, dan deploy pertama

Susunan: setiap rilis di foldernya sendiri, `current` menunjuk ke rilis yang aktif, dan `shared` menyimpan yang bertahan antar-deploy.

```
/var/www/sigrak/
├── current -> releases/20261031-1000
├── releases/
```

```
└── shared/
    ├── .env
    └── storage/
```

```
sudo -u sigrak mkdir -p /var/www/sigrak/releases
/var/www/sigrak/shared/storage/{app/public,framework/{cache,sessions,views},logs}
sudo chmod 750 /var/www/sigrak
sudo mkdir -p /var/log/sigrak && sudo chown sigrak:sigrak /var/log/sigrak
```

`.env` produksi (`/var/www/sigrak/shared/.env`, milik `sigrak`, mode 600). Hanya key yang terkait server yang dicantumkan; aplikasi menambahkan key-nya sendiri.

```
APP_NAME=SIGRAK
APP_ENV=production
APP_DEBUG=false
APP_KEY=                        # generated once with: php artisan key:generate --show
APP_URL=https://app.sigrak.com
APP_LOCALE=id
APP_FALLBACK_LOCALE=id

LOG_CHANNEL=stack
LOG_STACK=daily
LOG_DAILY_DAYS=14
LOG_LEVEL=warning

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=sigrak
DB_USERNAME=sigrak
DB_PASSWORD=<DB_PASSWORD>

REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
REDIS_PASSWORD=<REDIS_PASSWORD>

QUEUE_CONNECTION=redis
CACHE_STORE=redis
SESSION_DRIVER=redis
SESSION_SECURE_COOKIE=true
# SESSION_DOMAIN stays unset: app and exam site must not share cookies

TELESCOPE_ENABLED=false

# spatie/laravel-pdf v2 Gotenberg driver (config/laravel-pdf.php reads these keys)
LARAVEL_PDF_DRIVER=gotenberg
GOTENBERG_URL=http://127.0.0.1:3000
GOTENBERG_USERNAME=<GOTENBERG_USER>
GOTENBERG_PASSWORD=<GOTENBERG_PASSWORD>
```

`APP_KEY` sangat penting. Kunci Gemini milik guru dienkripsi dengannya (D-012). Kehilangan `APP_KEY` membuat setiap kunci AI yang tersimpan tidak bisa dibaca. Simpan salinannya di pengelola kata sandi, di luar server dan di luar cadangan.

Skrip deploy (dijalankan sebagai `sigrak`). Zona waktu aplikasi `Asia/Jakarta` diatur di `config/app.php`, bukan di sini.

```
sudo tee /usr/local/bin/sigrak-deploy > /dev/null <<'EOF'
#!/usr/bin/env bash
```

```
# Usage: sigrak-deploy <git-ref>   (run as the sigrak user)
set -euo pipefail
REF="${1:?git ref required}"
BASE=/var/www/sigrak
REL="$BASE/releases/$(date +%Y%m%d-%H%M%S)"

git clone --depth 1 --branch "$REF" git@github.com:arumiforge/sinaubareng.git "$REL"
cd "$REL"
rm -rf storage && ln -s "$BASE/shared/storage" storage
ln -s "$BASE/shared/.env" .env

composer install --no-dev --optimize-autoloader --no-interaction
# Assets: either public/build/ comes from CI, or build here when Node is installed:
[ -d public/build ] || { npm ci && npm run build && rm -rf node_modules; }

php artisan migrate --force
php artisan optimize
php artisan storage:link

ln -sfn "$REL" "$BASE/current"
sudo /usr/bin/systemctl reload php8.5-fpm
php artisan horizon:terminate
sudo /usr/bin/supervisorctl restart sigrak-scheduler

ls -1dt "$BASE"/releases/* | tail -n +6 | xargs -r rm -rf   # keep 5 releases
EOF
sudo chmod 755 /usr/local/bin/sigrak-deploy

sudo tee /etc/sudoers.d/sigrak-deploy > /dev/null <<'EOF'
sigrak ALL=(root) NOPASSWD: /usr/bin/systemctl reload php8.5-fpm, /usr/bin/supervisorctl restart
sigrak-scheduler
EOF
sudo chmod 440 /etc/sudoers.d/sigrak-deploy && sudo visudo -c
```

Server menarik kode dari GitHub dengan **deploy key** yang hanya-baca: `sudo -u sigrak ssh-keygen -t ed25519 -N "" -f /var/www/sigrak/.ssh/id_ed25519`, lalu tambahkan public key-nya di pengaturan repositori bagian Deploy keys (hanya-baca). Rollback: arahkan `current` ke folder rilis sebelumnya dan jalankan tiga perintah terakhir dari skrip.

11. Supervisor: Horizon dan scheduler

Horizon menjalankan semua worker antrean. Antrean dan jumlah worker-nya ada di `config/horizon.php` milik aplikasi (antrean `ai` mendapat supervisor sendiri, lihat `ai-provider-engine.md`). Supervisor hanya menjaga kedua proses tetap hidup.

```
sudo tee /etc/supervisor/conf.d/sigrak.conf > /dev/null <<'EOF'
[program:sigrak-horizon]
command=/usr/bin/php /var/www/sigrak/current/artisan horizon
user=sigrak
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/log/sigrak/horizon.log
stopwaitsecs=600

[program:sigrak-scheduler]
command=/usr/bin/php /var/www/sigrak/current/artisan schedule:work
user=sigrak
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/log/sigrak/scheduler.log
```

```
stopwaitsecs=60
EOF
sudo supervisorctl reread && sudo supervisorctl update
sudo supervisorctl status
```

`schedule:work` di bawah Supervisor menggantikan baris cron yang biasa (`* * * * * php artisan schedule:run`), yaitu bentuk yang ditunjukkan `architecture.md`. Keduanya menjalankan jadwal yang sama; pakai salah satu saja.

`stopwaitsecs=600` lebih lama dari langkah AI terlama (5 menit per langkah, draf D-022), sehingga deploy membiarkan job yang sedang berjalan selesai. Jaga `retry_after` antrean di atas timeout job terlama, sesuai syarat dokumen mesin AI.

Titik awal untuk `config/horizon.php` di VPS 8 vCPU / 16 GB (disetel ulang setelah ada beban nyata):

Supervisor	Antrean	Proses	Timeout
`supervisor-ai`	`ai`	4–12 (auto balance)	330 dtk
`supervisor-exports`	`exports`	4	180 dtk
`supervisor-default`	`default`	3	60 dtk

Ekspor memanggil Gotenberg; empat sekaligus muat dalam batas container 2 GB. Worker AI sebagian besar menunggu Gemini, jadi jumlahnya dibatasi kuota paket gratis guru dan corong 2-per-guru, bukan oleh CPU.

12. Cadangan dan uji pemulihan

Yang dicadangkan, setiap hari pukul 01.30 WIB (18.30 UTC):

- MySQL: `mysqldump` yang konsisten dari `sigrak`.
- Berkas: `shared/storage/app` (unggahan seperti kop, berkas yang dihasilkan).
- Tidak masuk cadangan: `.env` dan `APP_KEY` (disimpan di pengelola kata sandi, lihat bagian 10).

Salinan lokal disimpan 14 hari, salinan di luar server 30 hari. Tujuan di luar server adalah **Cloudflare R2**, dijangkau lewat `rclone` dengan remote `crypt`, sehingga berkas dienkripsi sebelum diunggah.

Di dasbor Cloudflare: R2 → Create bucket `<R2_BUCKET>` (lokasi: otomatis, privat). Lalu R2 → Manage API tokens → Create token dengan **Object Read & Write**, dibatasi ke bucket itu saja. Salin Access Key ID dan Secret Access Key; Account ID ada di halaman ringkasan R2. Simpan kedua kata sandi crypt di pengelola kata sandi: tanpanya salinan di luar server tidak bisa dibaca.

```
sudo -v ; curl -fsSL https://rclone.org/install.sh | sudo bash
rclone version | head -1    # must be 1.59 or newer for R2
sudo rclone config create offsite s3 provider=Cloudflare \
  access_key_id=<R2_ACCESS_KEY_ID> secret_access_key=<R2_SECRET_ACCESS_KEY> \
  endpoint=https://<R2_ACCOUNT_ID>.r2.cloudflarestorage.com region=auto acl=private
no_check_bucket=true
sudo rclone config create offsite-crypt crypt remote=offsite:<R2_BUCKET>/production \
  password=<BACKUP_CRYPT_PASSWORD> password2=<BACKUP_CRYPT_SALT> --obscure
sudo rclone lsd offsite:<R2_BUCKET> && echo 'R2 reachable'

sudo install -d -m 700 /etc/sigrak /var/backups/sigrak
sudo tee /etc/sigrak/backup.cnf > /dev/null <<'EOF'
[client]
user=backup
password=<DB_BACKUP_PASSWORD>
EOF
sudo chmod 600 /etc/sigrak/backup.cnf
```

```
sudo tee /usr/local/sbin/sigrak-backup > /dev/null <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
umask 077
. /etc/sigrak/monitoring.env
HC="https://hc-ping.com/${HC_PING_KEY}"
curl -fsS -m 10 --retry 2 -o /dev/null "$HC/backup/start"
trap 'curl -fsS -m 10 --retry 2 -o /dev/null --data-raw "backup failed at line $LINENO"
"$HC/backup/fail"' ERR
STAMP="$(date +%Y%m%d-%H%M)"
DIR=/var/backups/sigrak

mysqldump --defaults-extra-file=/etc/sigrak/backup.cnf --single-transaction --quick \
  --no-tablespaces --triggers --events --hex-blob --set-gtid-purged=OFF sigrak \
  | gzip > "$DIR/db-$STAMP.sql.gz"
tar -C /var/www/sigrak/shared -czf "$DIR/storage-$STAMP.tar.gz" storage/app

gzip -t "$DIR/db-$STAMP.sql.gz"
tar -tzf "$DIR/storage-$STAMP.tar.gz" > /dev/null

rclone copy "$DIR" offsite-crypt:daily --include "*-$STAMP.*"
rclone delete offsite-crypt:daily --min-age 30d
find "$DIR" -type f -mtime +14 -delete
curl -fsS -m 10 --retry 2 -o /dev/null "$HC/backup"
echo "backup $STAMP ok"
EOF
sudo chmod 700 /usr/local/sbin/sigrak-backup
echo '30 18 * * * root /usr/local/sbin/sigrak-backup >> /var/log/sigrak/backup.log 2>&1' | sudo tee
/etc/cron.d/sigrak-backup
sudo /usr/local/sbin/sigrak-backup    # first run by hand
```

Skrip ini mengirim ping ke Healthchecks.io saat mulai, saat berhasil, dan saat ada langkah yang gagal, sehingga cadangan yang gagal atau terlewat memicu peringatan (D-136). `/etc/sigrak/monitoring.env` berasal dari `server-monitoring.md` (di `docs/final/`).

Uji pemulihan — setiap bulan, dan setelah setiap perubahan pada bagian ini. Cadangan baru dianggap ada setelah pernah dipulihkan sekali. Di server, ke database sementara:

```
LATEST="$(ls -1t /var/backups/sigrak/db-*.sql.gz | head -1)"
sudo mysql -uroot -p -e "CREATE DATABASE sigrak_restore_test CHARACTER SET utf8mb4 COLLATE
utf8mb4_0900_ai_ci"
gunzip -c "$LATEST" | sudo mysql -uroot -p sigrak_restore_test
sudo mysql -uroot -p -e "
  SELECT table_name, table_rows FROM information_schema.tables
  WHERE table_schema IN ('sigrak','sigrak_restore_test') ORDER BY table_name, table_schema;"
sudo mysql -uroot -p -e "DROP DATABASE sigrak_restore_test"

mkdir -p /tmp/restore-test && tar -xzf "$(ls -1t /var/backups/sigrak/storage-*.tar.gz | head -1)" -C
/tmp/restore-test
du -sh /tmp/restore-test/storage/app /var/www/sigrak/shared/storage/app && rm -rf /tmp/restore-test
```

Jumlah baris adalah perkiraan (`table_rows`), jadi bandingkan secara kasar; untuk tabel penting (`users`, `schools`, dokumen yang dihasilkan) jalankan `SELECT COUNT(*)` di keduanya. Dua kali setahun, pulihkan juga dari salinan **di luar server** (`rclone copy offsite-crypt:daily/<file> /tmp/`) untuk membuktikan kata sandi enkripsinya berfungsi. Catat tanggal dan hasil setiap uji di log operasional.

Pemulihan bencana penuh: bangun ulang VPS dengan bagian 1–11, pulihkan dump ke `sigrak`, ekstrak `storage/app` ke `shared/storage/`, kembalikan `.env` dengan `APP_KEY` yang **asli**, lakukan deploy, lalu salin `/etc/cloudflared/` (konfigurasi dan JSON kredensial) ke VPS baru dan jalankan `cloudflared` di sana. Tidak

perlu mengubah DNS, karena hostname menunjuk ke tunnel, bukan ke IP.

13. Rotasi log

Log	Rotasi
Laravel (`storage/logs/laravel-YYYY-MM-DD.log`)	Oleh Laravel sendiri: `LOG_STACK=daily`, `LOG_DAILY_DAYS=14`
Nginx, MySQL, Redis, UFW	Aturan logrotate bawaan paket Ubuntu (biarkan)
`/var/log/sigrak/*.log` (Horizon, scheduler, cadangan)	Aturan di bawah
Docker (Gotenberg)	`daemon.json`: 3 × 10 MB per container
Journal systemd	Dibatasi di bawah

```
sudo tee /etc/logrotate.d/sigrak > /dev/null <<'EOF'
/var/log/sigrak/*.log {
    weekly
    rotate 8
    missingok
    notifempty
    compress
    delaycompress
    copytruncate
    su root sigrak
}
EOF
sudo logrotate -d /etc/logrotate.d/sigrak    # dry run, check for errors

sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nSystemMaxUse=500M\n' | sudo tee /etc/systemd/journald.conf.d/size.conf
sudo systemctl restart systemd-journald
```

`copytruncate` dipakai karena Supervisor tetap membuka berkas lognya.

14. Pemeriksaan akhir (produksi)

```
php -v | head -1                                # PHP 8.5.12+
mysql --version                                 # 8.4.x
redis-server --version                          # 8.2.x
sudo docker ps --format '{{.Image}} {{.Ports}}'    # gotenberg 8.39.0-chromium on 127.0.0.1:3000
sudo ss -tlnp | grep -E ':(3306|6379|3000|33060)\b' # all on 127.0.0.1 / ::1 only; no 33060
sudo ss -tlnp | grep -E ':(80|443|8080)\b'          # only 127.0.0.1:8080 (nginx)
sudo ufw status verbose                         # active, deny incoming, no ALLOW rules
systemctl is-active cloudflared                 # active
sudo cloudflared tunnel info sigrak-prod        # connections listed
sudo supervisorctl status                       # horizon + scheduler RUNNING
curl -sI https://app.sigrak.com | head -1       # 200 or 302 to login
curl -sI https://admin.sigrak.com | head -1     # 302 to the Cloudflare Access login
```

Di aplikasi: masuk, buat satu dokumen, ekspor ke PDF, dan periksa di Horizon bahwa antrean `ai` dan `exports` menjalankan job-nya. Buka `ujian.sigrak.com` di ponsel dan pastikan yang tampil adalah halaman masuk ujian, tanpa apa pun dari aplikasi utama. Pastikan Horizon hanya terbuka di `admin.sigrak.com` setelah login staf, dan Telescope tidak bisa dijangkau.

15. Staging (situs kedua di VPS yang sama)

Staging berjalan **di VPS produksi** sebagai situs kedua yang sepenuhnya terpisah. Staging berbagi layanan (MySQL, Redis, Gotenberg, Nginx, `cloudflared`) tetapi punya pengguna, kode, pool PHP-FPM, database, key Redis, worker, dan hostname sendiri. Setiap rilis aplikasi masuk ke staging lebih dulu.

Konsekuensinya. Karena layanannya dipakai bersama, pembaruan PHP, MySQL, Redis, Nginx, image Gotenberg, atau OS tidak bisa dicoba dulu di staging. Staging hanya menguji rilis aplikasi. Latih pembaruan layanan yang besar di VM sementara atau di snapshot VPS.

Hal	Produksi	Staging
Pengguna sistem	`sigrak`	`sigrak-staging`
Folder aplikasi	`/var/www/sigrak`	`/var/www/sigrak-staging` (susunan `releases/`, `shared/`, `current` yang sama)
Pool PHP-FPM	`sigrak`, 60 children	`sigrak-staging`, 10 children
MySQL	DB `sigrak`, pengguna `sigrak`	DB `sigrak_staging`, pengguna `sigrak_staging` (tanpa hak atas `sigrak`)
Redis	DB 0–3, prefix bawaan	DB 8–11, prefix `sigrak_staging_`
Worker	`sigrak-horizon`, `sigrak-scheduler`	`sigrak-staging-horizon` (1 proses per antrean), `sigrak-staging-scheduler`
Hostname	`app.`, `ujian.`, `admin.sigrak.com`	`staging-app.`, `staging-ujian.`, `staging-admin.sigrak.com`
Access	`ssh.` dan `admin.`	Semua hostname `staging-*`, hanya tim
Log	`/var/log/sigrak/`	`/var/log/sigrak-staging/`
Cadangan	Setiap hari ke R2	Tidak ada (hanya data seed)

Pengguna, folder, dan pool PHP-FPM:

```
sudo adduser --system --group --home /var/www/sigrak-staging --shell /bin/bash sigrak-staging
sudo usermod -aG sigrak-staging www-data
sudo -u sigrak-staging mkdir -p /var/www/sigrak-staging/releases \
  /var/www/sigrak-staging/shared/storage/{app/public,framework/{cache,sessions,views},logs}
sudo chmod 750 /var/www/sigrak-staging
sudo mkdir -p /var/log/sigrak-staging && sudo chown sigrak-staging:sigrak-staging /var/log/sigrak-staging

sudo tee /etc/php/8.5/fpm/pool.d/sigrak-staging.conf > /dev/null <<'EOF'
[sigrak-staging]
user = sigrak-staging
group = sigrak-staging
listen = /run/php/php8.5-fpm-sigrak-staging.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660
pm = dynamic
pm.max_children = 10
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
pm.max_requests = 500
request_terminate_timeout = 65s
```

```
EOF
sudo systemctl reload php8.5-fpm
```

Database dan pengguna. Pengguna staging hanya bisa melihat `sigrak_staging`:

```
sudo mysql -uroot -p <<'EOF'
CREATE DATABASE sigrak_staging CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE USER 'sigrak_staging'@'localhost' IDENTIFIED BY '<STAGING_DB_PASSWORD>';
GRANT ALL PRIVILEGES ON sigrak_staging.* TO 'sigrak_staging'@'localhost';
FLUSH PRIVILEGES;
EOF
```

Nginx: satu snippet dan tiga vhost, semuanya di `127.0.0.1:8080` seperti produksi, dengan `noindex`:

```
sed -e 's#/var/www/sigrak/current/public#/var/www/sigrak-staging/current/public#' \
    -e 's#php8.5-fpm-sigrak.sock#php8.5-fpm-sigrak-staging.sock#' \
    /etc/nginx/snippets/sigrak-common.conf | sudo tee /etc/nginx/snippets/sigrak-staging-common.conf >
/dev/null
echo 'add_header X-Robots-Tag "noindex, nofollow" always;' | sudo tee -a /etc/nginx/snippets/sigrak-
staging-common.conf > /dev/null

for site in app ujian admin; do
  sudo tee "/etc/nginx/sites-available/sigrak-staging-$site.conf" > /dev/null <<EOF
server {
    listen 127.0.0.1:8080;
    server_name staging-$site.sigrak.com;
    client_max_body_size 25m;
    access_log /var/log/nginx/sigrak-staging-$site.access.log;
    error_log  /var/log/nginx/sigrak-staging-$site.error.log;
    include snippets/sigrak-staging-common.conf;
}
EOF
  sudo ln -s "/etc/nginx/sites-available/sigrak-staging-$site.conf" /etc/nginx/sites-enabled/
done
sudo nginx -t && sudo systemctl reload nginx
```

Tunnel: tambahkan tiga hostname ke tunnel yang **sama**, di atas aturan penampung terakhir di `/etc/cloudflared/config.yml`. Tidak perlu hostname SSH terpisah, karena servernya sama.

```
  - hostname: staging-app.sigrak.com
    service: http://127.0.0.1:8080
  - hostname: staging-ujian.sigrak.com
    service: http://127.0.0.1:8080
  - hostname: staging-admin.sigrak.com
    service: http://127.0.0.1:8080
```

```
sudo cloudflared tunnel ingress validate --config /etc/cloudflared/config.yml
for h in staging-app staging-ujian staging-admin; do sudo cloudflared tunnel route dns sigrak-prod
"$h.sigrak.com"; done
sudo systemctl restart cloudflared
```

`tunnel route dns` butuh `cert.pem`; pulihkan dari pengelola kata sandi untuk langkah ini dan hancurkan lagi sesudahnya. Lalu tambahkan **satu** aplikasi Cloudflare Access yang mencakup ketiga hostname `staging-*`, hanya mengizinkan `<TEAM_EMAILS>`.

`.env` staging (`/var/www/sigrak-staging/shared/.env`, pemilik `sigrak-staging`, mode 600). Mulai dari daftar produksi di bagian 10 dan ubah key berikut. Prefix Redis mencegah Horizon staging mengambil job produksi:

```
APP_ENV=staging
APP_DEBUG=false
APP_KEY=                        # its own key; never reuse production's
APP_URL=https://staging-app.sigrak.com
LOG_LEVEL=debug

DB_DATABASE=sigrak_staging
DB_USERNAME=sigrak_staging
DB_PASSWORD=<STAGING_DB_PASSWORD>

REDIS_PREFIX=sigrak_staging_
REDIS_DB=8
REDIS_CACHE_DB=9
CACHE_PREFIX=sigrak_staging_
HORIZON_PREFIX=sigrak_staging_horizon:

MAIL_MAILER=log
TELESCOPE_ENABLED=true          # visible to staff only (architecture.md)
```

`noeviction` Redis berlaku global, jadi key cache staging juga butuh TTL. Gotenberg dipakai bersama (container dan kredensial yang sama). Gotenberg tidak menyimpan data, tetapi jaga ekspor staging tetap kecil.

Worker:

```
sudo tee /etc/supervisor/conf.d/sigrak-staging.conf > /dev/null <<'EOF'
[program:sigrak-staging-horizon]
command=/usr/bin/php /var/www/sigrak-staging/current/artisan horizon
user=sigrak-staging
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/log/sigrak-staging/horizon.log
stopwaitsecs=600

[program:sigrak-staging-scheduler]
command=/usr/bin/php /var/www/sigrak-staging/current/artisan schedule:work
user=sigrak-staging
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/log/sigrak-staging/scheduler.log
stopwaitsecs=60
EOF
sudo supervisorctl reread && sudo supervisorctl update
```

Di environment `config/horizon.php` milik staging (`staging`), setel 1 proses per antrean.

Deploy: salinan skrip bagian 10 dengan `BASE=/var/www/sigrak-staging` dan `sudo /usr/bin/supervisorctl restart sigrak-staging-scheduler`, dijalankan sebagai `sigrak-staging`. Satu baris sudoers seperti milik produksi, untuk `sigrak-staging`, mengizinkan reload FPM dan restart tersebut. Tambahkan `/var/log/sigrak-staging/*.log` ke aturan logrotate di bagian 13.

Aturan data. Staging hanya berisi data seed buatan. **Jangan pernah menyalin data produksi ke `sigrak_staging`**: produksi menyimpan nama siswa dan kunci AI guru yang terenkripsi. Tim memasukkan kunci Gemini uji miliknya sendiri lewat aplikasi staging.

Keputusan untuk dicatat

Nomornya diberikan saat integrasi.

1. **PHP 8.5** (≥ 8.5.12) dari `ppa:ondrej/php`, dengan PHP 8.4 sebagai cadangan hanya bila ada paket wajib yang menghalangi 8.5. Alasan: Laravel 13 mendukung 8.3–8.5, dan 8.5 punya masa dukungan terpanjang.
2. **MySQL 8.4 LTS** dari repositori APT Oracle (`mysql-8.4-lts`), bukan 8.0 bawaan Ubuntu (sudah habis masa dukungannya) dan bukan 9.7 LTS (D-007 menyebut MySQL 8).
3. **Redis 8.2** (rilis extended) dari packages.redis.io, dikunci ke `6:8.2.*`, dengan `noeviction` dan AOF aktif.
4. **Image Gotenberg** `gotenberg/gotenberg:8.39.0-chromium`: varian khusus Chromium, dipublikasikan di `127.0.0.1:3000` dengan basic auth dan batas memori 2 GB, di jaringan Docker sendiri yang koneksi keluar barunya dibuang; font di-mount dari `/opt/gotenberg/fonts`.
5. **Node 24 LTS hanya untuk build**, sebaiknya di CI; tanpa Node di server kecuali dibutuhkan.
6. **Paket Nginx Ubuntu** (1.24 di 24.04, ditambal Canonical), bukan build nginx.org, hanya mendengar di `127.0.0.1:8080`, HTTP biasa di balik tunnel.
7. **Tanpa IP publik; Cloudflare Tunnel untuk semua lalu lintas** (pemilik, 2026-10-31): satu tunnel `cloudflared` yang dikelola lokal per server merutekan `app.`, `ujian.`, `admin.`, dan `ssh.` ke localhost. IP sebenarnya dari `CF-Connecting-IP` (hanya dipercaya dari 127.0.0.1). UFW menolak semua koneksi masuk. SSH dan `admin.` ada di balik Cloudflare Access. Ini menggantikan rancangan Origin CA / Authenticated Origin Pulls / daftar izin IP Cloudflare.
8. **Tiga vhost Nginx untuk satu codebase Laravel** (`app.`, `ujian.`, `admin.`), seperti di `architecture.md`, dengan cookie sesi host-only (tanpa `SESSION_DOMAIN` bersama).
9. **Deploy berbasis folder rilis** (`releases/`, `shared/`, symlink `current`), menyimpan 5 rilis; PHP-FPM di-reload pada setiap deploy karena OPcache tidak memeriksa timestamp.
10. **Supervisor menjalankan Horizon dan** `schedule:work` (tanpa entri cron untuk scheduler; `architecture.md` menunjukkan bentuk cron yang setara); jumlah worker ada di `config/horizon.php`.
11. **Cadangan:** setiap hari 01.30 WIB (18.30 UTC) `mysqldump` + tarball `storage/app`, 14 hari lokal, 30 hari di luar server lewat remote `rclone` yang terenkripsi; uji pemulihan setiap bulan, pemulihan dari luar server dua kali setahun. `APP_KEY` disimpan di pengelola kata sandi, tidak pernah di cadangan.
12. **Server dan PHP berjalan dalam UTC; aplikasi berjalan dalam WIB** (`Asia/Jakarta`, D-165), dan sesi MySQL-nya memakai `+07:00` (seperti di `architecture.md`). Waktu cron di panduan ini dalam UTC.
13. **Domain** `sigrak.com` dengan hostname `app.`, `ujian.`, `admin.` (pemilik, 2026-10-31).
14. **Penyedia VPS: IDCloudHost Cloud VPS, Jakarta** (pemilik, 2026-10-31).
15. **Cadangan di luar server di Cloudflare R2**, token Object Read & Write yang dibatasi ke bucket, dienkripsi dengan remote crypt `rclone` (pemilik, 2026-10-31).
16. **Staging adalah situs kedua di VPS produksi** (pemilik, 2026-10-31). Staging punya pengguna, pool FPM, DB dan pengguna DB, prefix Redis, serta program Horizon/scheduler sendiri. Hostname `staging-*`-nya berjalan di tunnel yang sama di balik Cloudflare Access. MySQL, Redis, Gotenberg, Nginx, dan `cloudflared` dipakai bersama, jadi hanya rilis aplikasi yang diuji di staging. Hanya data seed.
17. **Key** `.env` **Gotenberg** mengikuti `spatie/laravel-pdf` v2: `LARAVEL_PDF_DRIVER=gotenberg`, `GOTENBERG_URL`, `GOTENBERG_USERNAME`, `GOTENBERG_PASSWORD`.
18. **Pemantauan server adalah tugas terpisah** (pemilik, 2026-10-31); tidak dibahas di panduan ini.
19. **Ubuntu Server 24.04 LTS** sebagai ganti 22.04 (pemilik, 2026-10-31; D-086).
20. **VPS produksi: paket kustom IDCloudHost, 8 vCPU / 16 GB RAM** (pemilik, 2026-10-31; D-087); ukuran di bagian 3–5, 7, dan 11 mengikutinya.
21. **Disk produksi: NVMe 320 GB** (pemilik, 2026-10-31).

Pertanyaan terbuka

- [x] IDCloudHost sudah memastikan internet keluar (NAT) termasuk TCP/UDP 7844 untuk VM tanpa IP publik (2026-10-31).
- [ ] `sigrak.com` sudah dipilih; pemeriksaan merek di PDKI dalam `product-concept.md` tetap berlaku untuk nama itu.

Dijawab pemilik pada 2026-10-31: R2 untuk cadangan di luar server, IDCloudHost Jakarta, domain `sigrak.com`, staging wajib, key `.env` Gotenberg ditetapkan sekarang (keputusan 17), pemantauan sebagai tugas terpisah, Ubuntu 24.04 LTS, paket kustom 8 vCPU / 16 GB, Cloudflare Tunnel karena VPS tanpa IP publik, disk produksi 320 GB, staging sebagai situs kedua di VPS yang sama.